

Email Verification with Resend Code Limit (Max 3 Attempts)

Overview

This feature implements email-based verification code delivery with a maximum resend limit of 3 attempts, replacing the previous terminal-based verification code display.

Changes Made

1. Database Schema

File: `src/models/db_models.py`

Added new column to `PendingRegistration` model:

```
resend_attempts = Column(Integer, default=0)
```

This tracks how many times a user has requested a code resend.

2. Backend Services

VerificationRepo (`src/repositories/verification_repo.py`)

Added three new methods:

- `increment_resend_attempts()` - Increments resend counter
- `update_code()` - Updates verification code when resending (resets attempts to 0)
- Enhanced `create_or_update()` - Initializes `resend_attempts` to 0

AuthService (`src/services/auth_service.py`)

Added constant:

```
MAX_RESEND_ATTEMPTS = 3
```

Added new method:

```
resend_verification_code(db, email) -> dict
```

This method:

- Validates pending registration exists
- Checks resend attempts limit (max 3)
- Generates new 6-digit code
- Updates code with fresh expiry time (15 minutes)
- Resets code verification attempts to 0
- Increments resend_attempts counter
- Sends code via email
- Returns remaining resends count

3. API Endpoints

New Endpoint: POST `/api/auth/register/resend-code`

Request:

```
{  
  "email": "user@example.com"  
}
```

Response (Success):

```
{  
  "message": "Verification code resent to user@example.com. Code is valid for 15 minutes.",  
  "resend_attempts_remaining": 2  
}
```

Response (Max Attempts Exceeded):

```
{  
  "detail": "Maximum resend attempts (3) exceeded. Please request a new registration."  
}
```

4. Frontend UI Updates

File: `static/index.html`

State Variables Added:

```
const [resendAttempts, setResendAttempts] = useState(0);  
const [resendCooldown, setResendCooldown] = useState(0);
```

Handler Function Added:

```
handleResendCode() async
```

Features:

- Validates max attempts not exceeded
- Implements 60-second cooldown between resends
- Updates UI with remaining attempts
- Shows loading state during request
- Displays success/error messages
- Auto-clears verification code input on resend
- Countdown timer in button text

UI Changes:

1. **Removed** Mock Mode notification about checking terminal console
2. **Added** Resend Code button with:
 - Disabled state when max attempts reached
 - Countdown timer display (e.g., "Resend in 45s")
 - Attempt counter (e.g., "Resend Code (2/3)")
 - Loading state with spinner
 - Color-coded states:
 - Blue: Primary verify button
 - Gray/Disabled: When max attempts reached
 - Gray/Disabled: During cooldown

Button States:

- **Default:** "Resend Code (0/3)" - Ready to use
- **Sending:** "Sending..." - Shows spinner during request
- **Cooldown:** "Resend in 60s" - Countdown active
- **Max Attempts:** "Resend Code (3/3)" - Disabled and grayed out

Security Features

1. **Attempt Limiting:** Maximum 3 resends per registration request
2. **Rate Limiting:** 60-second cooldown between resend requests
3. **Code Expiry:** Codes valid for 15 minutes (can be configured)
4. **Attempt Reset:** Verification attempts reset to 0 on code resend
5. **Email Verification:** Prevents unauthorized email changes through rate limiting
6. **Session Management:** No manual action needed; state is server-managed

Email Service Integration

The `EmailService.send_verification_code()` method supports:

- **SMTP (Gmail or Custom):** Uses SMTP_USERNAME, SMTP_PASSWORD from .env
- **Resend API:** Uses RESEND_API_KEY from .env
- **Fallback:** Terminal output when credentials not configured (dev mode)

Configuration in .env:

```
# SMTP Settings (Gmail)
SMTP_SERVER=smtp.gmail.com
SMTP_PORT=587
SMTP_USERNAME=your-email@gmail.com
SMTP_PASSWORD=your-app-password
EMAIL_FROM=support@sthxtechnologies.com

# Or Resend API
RESEND_API_KEY=your-resend-api-key
```

User Flow

1. Registration Form:

- User enters name, email, password, role
- Clicks "Send Verification Code"

2. Verification Screen:

- Code sent to user's email (instead of terminal)
- User enters 6-digit code
- Can click "Resend Code" up to 3 times
- 60-second cooldown between resends

3. Verification:

- User submits code
- If correct: Account created, redirects to login
- If incorrect: Error message with remaining attempts (max 5)

4. Resend Limits:

- 1st resend: "Resend Code (1/3)" + 60s cooldown
- 2nd resend: "Resend Code (2/3)" + 60s cooldown
- 3rd resend: "Resend Code (3/3)" - Button disabled
- 4th attempt: Error "Maximum resend attempts (3) exceeded"

Error Handling

Scenario	Status	Message
Code sent	200	"Verification code sent to..."
No pending registration	400	"No pending registration request found"
Max resends exceeded	400	"Maximum resend attempts (3) exceeded"
Invalid email	400	"User with this email already exists"
Weak password	400	"Password must be at least 8 characters..."
Network error	500	Error details

Constants

Constant	Value	Purpose
<code>MAX_OTP_ATTEMPTS</code>	5	Max wrong code entries
<code>MAX_RESEND_ATTEMPTS</code>	3	Max resend requests
<code>OTP_EXPIRY_MINUTES</code>	15	Code validity duration
<code>RESEND_COOLDOWN</code>	60s	Cooldown between resends

Database Migrations

If upgrading from previous version:

```
ALTER TABLE pending_registrations ADD COLUMN resend_attempts INTEGER DEFAULT 0;
```

Or create new table:

```
CREATE TABLE pending_registrations (  
  id INTEGER PRIMARY KEY,  
  email VARCHAR UNIQUE,  
  name VARCHAR,  
  password_hash VARCHAR,  
  role VARCHAR DEFAULT 'receptionist',  
  code VARCHAR,  
  attempts INTEGER DEFAULT 0,  
  resend_attempts INTEGER DEFAULT 0,  
  expires_at DATETIME,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

Testing

Manual Testing Checklist

- ☐ Send verification code to email
- ☐ Resend code and verify new code received
- ☐ Attempt resend 3 times, verify 4th is blocked

- ☐ Verify 60-second cooldown works
- ☐ Submit wrong code 5 times, verify rejection
- ☐ Submit correct code and create account
- ☐ Verify email notifications contain correct code
- ☐ Test with SMTP (Gmail)
- ☐ Test with Resend API
- ☐ Test with fallback (mock mode)

API Testing

```
# Request verification code
curl -X POST http://localhost:8000/api/auth/register/request-code \
  -H "Content-Type: application/json" \
  -d '{
    "name": "John Doe",
    "email": "john@example.com",
    "password": "SecurePass123!",
    "role": "receptionist"
  }'

# Resend verification code
curl -X POST http://localhost:8000/api/auth/register/resend-code \
  -H "Content-Type: application/json" \
  -d '{"email": "john@example.com"}'

# Verify and register
curl -X POST http://localhost:8000/api/auth/register/verify \
  -H "Content-Type: application/json" \
  -d '{"email": "john@example.com", "code": "123456"}'
```

Future Enhancements

- ☐ Customizable resend limit (admin setting)
- ☐ Customizable code expiry time
- ☐ SMS OTP support
- ☐ Biometric verification option
- ☐ Magic link verification (email link instead of code)
- ☐ Resend attempt recovery (admin can reset)
- ☐ Verification code history/audit log

- ☐ Email template customization
- ☐ Multi-language support

Troubleshooting

Code not received in email

1. Check `.env` has SMTP or RESEND credentials configured
2. Check logs for email service errors
3. In dev mode (credentials not set), code prints to terminal
4. Check spam/junk folders
5. Verify email address is correct

"Maximum resend attempts exceeded" immediately

1. Check database for existing pending_registration record
2. Clear old pending registration if user starts over
3. Admin can manually reset resend_attempts in DB

Cooldown not working

1. Verify frontend has JavaScript enabled
2. Check browser console for errors
3. Verify 60-second timer is running in UI

Code expired

- User has 15 minutes to enter code
- Resend generates new code with fresh 15-minute timer
- Previous codes become invalid when new code is sent